

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node
5  {
6      int data;
7      struct node *next;
8  };
9
10 struct node *head = NULL;
11
12 void insertAtfirst()
13 {
14     int value;
15     struct node *newnode, *temp;
16     newnode = (struct node *) malloc (sizeof(struct node));
17     if (newnode == NULL) {
18         printf("memory insufficient\n");
19         return;
20     }
21     printf("enter the data: ");
22     scanf("%d",&value);
23     newnode ->data = value;
24     newnode ->next = NULL;
25     if (head == NULL) {
26         head = newnode;
27         newnode->next = head;
28     }
29     else {
30         temp = head;
31         while (temp->next != head) {
32             temp = temp->next;
33         }
34         temp->next = newnode;
35         newnode->next = head;
36         head = newnode;
37     }
38     printf("%d is inserted at first\n",value);
39 }
40
41 void insertAtlast()
42 {
43     int value;
44     struct node *temp, *newnode;
45     newnode = (struct node *) malloc (sizeof(struct node));
46     if(newnode == NULL) {
47         printf("insufficient memory");
48         return;
49     }
50     printf("enter the data: ");
51     scanf("%d",&value);
52     newnode ->data = value;
53     newnode ->next = NULL;
54     if (head == NULL) {
55         head = newnode;
56     }
57     else {
58         temp = head;
59         while (temp->next != head) {
60             temp = temp->next;
61         }
62         temp ->next = newnode;
63         newnode ->next = head;
64     }
65     printf("%d is inserted at lest\n",value);
66 }

```

```

67
68 void insertAtpos()
69 {
70     int value, pos;
71     struct node *temp, *newnode;
72     newnode = (struct node *) malloc (sizeof(struct node));
73     if(newnode == NULL) {
74         printf("insufficient memory");
75         return;
76     }
77     printf("enter the data: ");
78     scanf("%d",&value);
79     newnode ->data = value;
80     newnode ->next = NULL;
81     printf("enter the position: ");
82     scanf("%d",&pos);
83     if (head == NULL) {
84         head = newnode;
85     }
86     else {
87         temp = head;
88         for (int i=2;i<pos;i++){
89             temp = temp->next;
90         }
91         newnode ->next = temp->next;
92         temp ->next = newnode;
93     }
94     printf("%d is inserted at %d position\n");
95 }
96
97 void deleteFirst()
98 {
99     struct node *temp;
100    if (head == NULL) {
101        printf("list is empty\n");
102        return;
103    }
104    if (head ->next == head) {
105        head = NULL;
106    }
107    else {
108        temp = head;
109        while (temp ->next != head) {
110            temp = temp->next;
111        }
112        printf("%d is deleted",head->data);
113        temp ->next = head ->next;
114        head = head ->next;
115    }
116 }
117
118 void deletePos()
119 {
120     int pos;
121     struct node *temp, *temp2;
122     if (head == NULL) {
123         printf("list is empty");
124     }
125     else {
126         printf("enter the position: ");
127         scanf("%d",&pos);
128         temp = head;
129         for (int i=2;i<pos;i++) {
130             temp = temp ->next;
131         }
132         temp2 = temp->next;

```

```

133     printf("%d is deleted", temp2->data);
134     temp->next = temp2->next;
135     free(temp2);
136 }
137 }
138
139 void display()
140 {
141     struct node *temp;
142
143     if (head == NULL) {
144         printf("List is empty\n");
145         return;
146     }
147     temp = head;
148     while (temp->next != head) {
149         printf("%d ", temp->data);
150         temp = temp->next;
151     }
152     printf("%d\n", temp->data);
153 }
154
155 int main()
156 {
157     int ch;
158     printf("Linked List Menu\n");
159     printf("1. InsertAtFirst\n");
160     printf("2. InsertAtLast\n");
161     printf("3. InsertAtSpecifiedPosition\n");
162     printf("4. DeleteAtFirst\n");
163     printf("5. DeleteAtSpecifiedPositon\n");
164     printf("6. Display\n");
165     printf("7. Exit\n");
166     while (1)
167     {
168         printf("Enter your choice (1/2/3/4/5/6/7): ");
169         scanf("%d", &ch);
170         switch (ch)
171         {
172             case 1:
173                 insertAtfirst();
174                 break;
175             case 2:
176                 insertAtlast();
177                 break;
178             case 3:
179                 insertAtpos();
180                 break;
181             case 4:
182                 deleteFirst();
183                 break;
184             case 5:
185                 deletePos();
186                 break;
187             case 6:
188                 display();
189                 break;
190             case 7:
191                 return 0;
192             default:
193                 printf("your entered wrong choice\n");
194         }
195     }
196     return 0;
197 }
198

```

199
200
201